From Statistical Code Generation to Correct-by-Construction AI Programming: A Basis-Function Composition Framework

Yu Wu✉
wuyu14@mails.ucas.edu.cn, yuwu@cdut.edu.cn
College of Management Science, Chengdu University of Technology
Chengdu, China

Yuchen Yang
go98pif@tum.de
School of Computation, Information and Technology, Technical University of Munich
Munich, Germany

Abstract

Large language models have significantly advanced AI-assisted programming, but current systems still rely mainly on statistical code generation, execution feedback, testing, and human review. These mechanisms improve practical usability, yet they do not by themselves provide a construction process in which program correctness can be structurally organized, propagated, and checked. This paper presents a basis-function composition framework for correct-by-construction AI programming. The framework represents a program not merely as a sequence of generated code tokens, but as a structured composition of basis functions equipped with explicit specifications, preconditions, postconditions, and correctness evidence. Functional positions, candidate basis-function sets, hierarchical expansion, and correctness-preserving composition operators are introduced to organize automated program construction at multiple levels of abstraction. Within this framework, large language models are repositioned as assistants for specification interpretation, candidate retrieval, structural search, and code realization, rather than as the source of correctness itself. We illustrate the framework using the 0–1 knapsack problem, showing how basis-function composition supports dynamic-programming construction and helps expose semantic mismatches such as invalid update orders in space-optimized implementations. The paper argues that verifiable AI programming requires a shift from generating plausible code strings to constructing structured, interpretable, and checkable program objects.

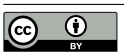
CCS Concepts

• **Software and its engineering → Formal software verification**.

Keywords

AI programming, large language models, correct-by-construction, program construction, compositional verification, basis-function composition

ACM Reference Format:
Yu Wu and Yuchen Yang. 2026. From Statistical Code Generation to Correct-by-Construction AI Programming: A Basis-Function Composition Framework.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASYDE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2989-8/2026/10
https://doi.org/10.1145/3843777.3844528

In *Proceedings of the 8th International Workshop on Automated and verifiable Software sYstem DEvelopment (ASYDE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3843777.3844528

1 Introduction

Large language models (LLMs) have rapidly changed the practice of software development. They can generate functions from natural language descriptions, complete code fragments, repair bugs, produce tests, explain programs, and interact with external tools. Recent agentic software engineering systems further extend these capabilities by planning, executing code, observing failures, and iteratively revising their outputs. These advances make AI-assisted programming increasingly useful, but they also raise a central question for automated and verifiable software development: how can AI systems construct programs whose correctness is not only tested after generation, but organized during construction?

Current LLM-centered programming systems still rely largely on statistical code generation followed by execution feedback, testing, self-repair, or human review. These mechanisms improve practical usability, but they do not by themselves explain why a generated program is correct. A program may be syntactically valid, stylistically natural, and able to pass a finite set of tests, while still violating the intended specification on untested inputs. This limitation is especially important for algorithmic programs, optimization routines, high-reliability modules, and decision-making software, where correctness requires more than empirical success on selected examples.

The key limitation is structural. Statistical code generation treats a program mainly as a token sequence generated under a learned probability distribution. Program correctness, by contrast, is a semantic relation between a specification and a program: whenever the required precondition holds, execution should lead to the required postcondition. Testing and execution feedback can expose errors and guide repair, but they are primarily posterior mechanisms. They do not provide a construction process in which specifications, state dependencies, local correctness evidence, and composition conditions are explicitly maintained.

This paper proposes a basis-function composition framework for correct-by-construction AI programming. The framework represents a program not merely as a generated code string, but as a structured composition of basis functions. A basis function is a program construction unit equipped with an implementation, preconditions, postconditions, and supporting correctness evidence. Basis

functions are connected through correctness-preserving composition operators, such as sequential composition, conditional branching, loops, and recursion. These operators specify when local correctness can be propagated to global correctness.

The framework introduces three additional concepts for organizing automated program construction. A functional position represents a required functional role in the construction process, together with its input, output, precondition, and postcondition. A candidate basis-function set contains alternative basis functions that may satisfy the specification of a functional position. Hierarchical expansion allows a high-level basis function to be either invoked as a reusable unit or expanded into a lower-level composition structure when more detailed construction or verification is needed. Under this view, AI programming becomes a process of selecting, expanding, composing, and realizing specified basis functions under correctness-preserving conditions.

This perspective also clarifies the role of LLMs. In the proposed framework, an LLM is not treated as the source of correctness itself. Instead, it assists the construction process by interpreting specifications, proposing functional decompositions, retrieving and ranking candidate basis functions, exploring composition structures, and realizing checked structures as executable code. Correctness is supported by explicit specifications, local correctness evidence, and composition conditions. Thus, the target of AI programming shifts from producing plausible code sequences to constructing structured, interpretable, and checkable program objects.

We illustrate the framework using the 0–1 knapsack problem. The example shows how a high-level algorithmic task can be decomposed into functional positions such as state definition, initialization, state transition, loop organization, and result extraction. It also shows how correctness propagates through the construction chain: correct state semantics, correct base cases, correct transition, dependency-respecting loop order, and correct result extraction. The same example further shows how the framework can expose a semantic mismatch in a one-dimensional rolling-array implementation: an increasing capacity loop violates the dependency condition of the 0–1 transition and may incorrectly reuse the same item multiple times. Such an error is not a surface-level syntax problem, but a mismatch between a transition basis function and a loop operator.

This paper makes the following contributions:

- It formulates correct-by-construction AI programming as a shift from statistical code generation toward verifiable program construction.
- It proposes a basis-function composition framework that represents programs as structured compositions of specified and evidence-carrying construction units.
- It introduces functional positions, candidate basis-function sets, hierarchical expansion, and correctness-preserving composition operators as core elements of the construction process.
- It uses the 0–1 knapsack problem to illustrate how the framework supports correctness propagation and detects semantic mismatches that may be missed by direct code generation or limited testing.

The remainder of this paper is organized as follows. Section 2 positions the proposed framework with respect to related work. Section 3 presents the basis-function composition framework. Section 4 discusses correctness-preserving composition. Section 5 illustrates the framework using the 0–1 knapsack problem. Section 6 discusses limitations and future research directions. Section 7 concludes the paper.

2 Background and Positioning

The proposed framework is related to several lines of work in automated software engineering, including LLM-based code generation, agentic programming, program synthesis, formal verification, and component-based software engineering. This section briefly positions the paper with respect to these directions.

2.1 LLM-Based Code Generation and Agentic Programming

LLMs have substantially improved the ability to generate code from natural-language requirements, function signatures, examples, and surrounding context. Models trained on large code corpora can produce function-level programs, solve competitive-programming-style tasks, and support interactive programming workflows [2, 10, 16]. More recent agentic software engineering systems further extend this ability by allowing models to plan, call tools, execute code, inspect feedback, and revise their outputs over multiple steps [7, 25]. These developments make LLMs increasingly useful not only as code generators, but also as participants in broader software construction processes.

However, the reliability of such systems is still usually improved through posterior mechanisms such as execution feedback, unit tests, self-debugging, automated repair, or iterative refinement [3, 8, 11]. These mechanisms are valuable, but they do not change the basic object being produced: the system still often starts from generated code and then checks or repairs it. In contrast, this paper asks how the construction process itself can be organized so that specifications, functional roles, candidate structures, and correctness evidence are made explicit before or during code realization.

2.2 Program Synthesis and Verification-Oriented Development

Program synthesis provides a more specification-centered view of program generation. Instead of producing code only from statistical patterns, synthesis systems aim to construct programs satisfying formal specifications, examples, sketches, or syntactic constraints [1, 5, 17]. This line of work shows that program generation can be guided by explicit constraints rather than by surface-level code likelihood alone.

Formal verification provides another foundation for reliable programming. Hoare logic, design by contract, proof-carrying code, SMT solving, and verification tools such as Dafny show how program behavior can be specified, reasoned about, and checked using preconditions, postconditions, invariants, contracts, and logical constraints [4, 6, 9, 13, 14]. These ideas are central to correct-by-construction software development because they make correctness conditions explicit.

The proposed framework is inspired by these specification-based and verification-oriented traditions, but it has a different focus. It does not attempt to replace LLM-based programming with full formal synthesis or full formal verification. Instead, it introduces an intermediate construction layer in which LLMs may still assist with specification interpretation, candidate retrieval, structural search, and code realization, while correctness-relevant information is organized through basis functions and correctness-preserving composition operators.

2.3 Component-Based Construction

Component-based software engineering emphasizes modularity, reuse, encapsulation, interfaces, and composition [12, 15, 18]. This tradition is relevant because the proposed framework also treats programs as compositions of reusable units. However, a basis function is not merely a conventional software component. It is a construction unit equipped with an implementation, preconditions, postconditions, and supporting correctness evidence. Its composition with other basis functions is not only an interface-level connection, but also a correctness-preserving relation.

This distinction is important for LLM-assisted programming. Existing components are often reused by human developers under architectural and interface constraints. In the proposed framework, basis functions are selected from candidate sets associated with functional positions, may be expanded into lower-level structures when needed, and are composed under explicit correctness conditions. Thus, the framework extends component-based thinking toward specification-aware and verification-oriented AI programming.

2.4 Position of This Paper

The paper does not claim that correctness can be obtained from LLM generation alone. Nor does it claim that every generated program should be fully verified from scratch. Instead, it proposes a construction-oriented view: an AI programming system should construct a structured program object by decomposing a task into functional positions, selecting basis functions with explicit specifications and evidence, composing them through correctness preserving operators, and then realizing the constructed structure as code.

The contribution of this paper is not to replace existing formal methods, contract-based design, program synthesis, or compositional verification. Rather, it organizes these ideas around the emerging setting of LLM-assisted programming. In this setting, generative models may propose alternative decomposition structures for a given programming task, candidate basis functions for functional positions, and possible code realizations, while their acceptance must depend on explicit specifications, correctness evidence, and correctness-preserving composition conditions. Thus, LLMs, testing, synthesis, verification, and component reuse are treated as complementary mechanisms within a common construction process for correct-by-construction AI programming.

3 Basis-Function Composition Framework

This section presents the basis-function composition framework. The goal is to provide an intermediate construction layer between task specifications and generated code. Instead of asking an LLM to directly produce a complete code string, the framework first constructs a structured program object composed of specified basis functions and correctness-preserving composition operators.

3.1 Basic Construction View

Given a task specification, the framework organizes AI programming as the following construction process:

$$\text{Spec} \rightarrow \text{FP} \rightarrow \text{C} \rightarrow \text{Struct} \rightarrow \text{Code}. \tag{1}$$

where *Spec* denotes the task specification, *FP* denotes a set of functional positions obtained by decomposition, *C* denotes candidate basis-function sets for these positions, *Struct* denotes the constructed program structure, and *Code* denotes the final code realization.

This view changes the target of AI programming. The immediate target is not a code sequence, but a program structure constrained by specifications and composition conditions. Code generation becomes the final realization step after the main construction decisions have been made.

3.2 Basis Functions

A basis function is a program construction unit equipped with explicit semantic information. Formally, a basis function can be represented as

$$b = \langle Impl, Pre, Post, Evd \rangle, \tag{2}$$

where $Impl$ is the implementation, Pre is the precondition, $Post$ is the postcondition, and Evd is the evidence supporting the correctness of the basis function.

A basis function differs from an ordinary code fragment in two ways. First, it is not only executable but also specified. Its intended behavior is described by preconditions and postconditions. Second, it carries some form of correctness evidence, such as a formal proof, contract checking, static analysis, unit tests, or human review. The strength of evidence may vary across basis functions, but the evidence must be explicit enough to support construction-time reasoning.

For example, a sorting basis function may require a finite sequence of comparable elements as input and guarantee that the output is a nondecreasing permutation of the input. Different implementations, such as mergesort, quicksort, or insertion sort, may all satisfy this functional specification while differing in stability, time complexity, and space usage. They can therefore serve as alternative basis functions for the same functional role.

3.3 Functional Positions and Candidate Sets

A functional position represents a required role in a program construction process. It specifies what needs to be done at a certain point of the program structure, together with the relevant inputs, outputs, preconditions, and postconditions. Let

$$FP = \{fp_1, fp_2, \ldots, fp_n\} \tag{3}$$

be the set of functional positions decomposed from a task specification. For each functional position fp_i, the construction system

retrieves a candidate basis-function set from the basis-function library B:

$$C(fp_i) = \{b \in B \mid \mathrm{Pre}(fp_i) \Rightarrow \mathrm{Pre}(b),\ \mathrm{Post}(b) \Rightarrow \mathrm{Post}(fp_i)\}, \quad (4)$$

where the first condition means that the functional position provides enough information to invoke the basis function, and the second condition means that the result of the basis function is strong enough to satisfy the goal of the position.

In practice, the basis-function library is not assumed to be generated entirely by an LLM. It is a curated and incrementally extensible repository. Initial entries may come from verified libraries, contract-annotated components, standard algorithmic templates, proof-carrying modules, or human-reviewed reusable implementations. An LLM may propose new candidate basis functions, but such proposals are treated as untrusted candidates until their interfaces, preconditions, postconditions, dependencies, and evidence types have been checked by static analysis, testing, contract checking, formal verification, or human review.

The construction problem is therefore not direct code generation, but constrained selection from candidate sets. For each functional position, the system should select a suitable basis function:

$$b_i \in C(fp_i). \quad (5)$$

The selected basis functions form a valid program only when they can be connected by composition operators whose correctness preserving conditions are satisfied.

3.4 Composition Operators

Composition operators specify how basis functions can be connected. Let

$$\Omega = \{\omega_1, \omega_2, \ldots, \omega_k\} \quad (6)$$

be a set of composition operators. Typical operators include sequential composition, conditional branching, loop iteration, recursion, mapping, reduction, and selection among alternatives.

For sequential composition, suppose b_i is executed before b_j. The composition is valid only if the postcondition of b_i entails the precondition of b_j:

$$Post(b_i) \Rightarrow Pre(b_j). \quad (7)$$

This condition prevents the system from composing locally correct basis functions in an invalid way. For example, a module may produce a sorted list, while the next module may require a heap; or a state-transition step may require an initialized table that has not yet been constructed. Such errors are not necessarily syntax errors, but composition mismatches.

Other composition operators have their own correctness preserving conditions. Conditional branching requires that all branches be covered and lead to compatible postconditions. Loops require initialization, invariant preservation, dependency consistency, and termination. Recursion requires correct base cases, decreasing subproblem size, valid result combination, and termination. These conditions are discussed further in Section 4.

3.5 Hierarchical Expansion

Basis functions are not restricted to a fixed granularity. A high-level basis function can be invoked as a reusable unit when its specification and evidence are sufficient, or expanded into a lower-level composition structure when more detailed construction, inspection, or verification is needed.

Formally, a high-level basis function b^H may correspond to a lower-level structure S^L:

$$b^H \equiv S^L, \qquad S^L \in Gen(B^L, \Omega^L), \quad (8)$$

where B^L is a lower-level basis-function library and Ω^L is a lower-level set of composition operators.

This hierarchical view supports both reuse and checkability. For example, an LLM-assisted system may select a known algorithmic component, such as a shortest-path solver or a knapsack solver, and use it directly if the component is trusted and sufficiently specified. If more assurance is needed, the component can be expanded into lower-level functional positions, such as state definition, initialization, transition, iteration, and result extraction.

Hierarchical expansion may be proposed by an LLM, retrieved from the library, or specified by a human developer. However, the proposed expansion is accepted only if the lower-level functional positions collectively satisfy the specification of the parent basis function and the composition obligations among the lower-level basis functions are satisfied. Thus, the LLM proposes possible decompositions, while the construction framework checks specification coverage, dependency consistency, and evidence requirements.

3.6 Role of LLMs in the Framework

In the proposed framework, LLMs are not removed from the programming process. Instead, their role is constrained and redirected. They can assist with:

- interpreting natural-language specifications and extracting candidate preconditions and postconditions;
- proposing functional decompositions and identifying functional positions;
- retrieving, ranking, or adapting candidate basis functions;
- exploring alternative composition structures;
- translating a checked program structure into executable code.

However, correctness is not attributed to the LLM itself. Correctness depends on whether the selected basis functions satisfy their specifications and whether their composition satisfies correctness-preserving conditions. The LLM is therefore an assistant for construction, not the source of correctness.

3.7 Summary

The basis-function composition framework reformulates AI programming as structured construction under specification constraints. A program is constructed by decomposing a task into functional positions, selecting suitable basis functions from candidate sets, composing them through correctness-preserving operators, optionally expanding high-level functions into lower-level structures, and finally realizing the checked structure as code. This provides a foundation for moving from statistical code generation toward correct-by-construction AI programming.

4 Correctness-Preserving Construction

A program constructed from basis functions is not correct merely because each selected basis function is locally correct. Local correctness must be propagated through composition operators. This section summarizes the main correctness-preserving conditions used in the proposed framework.

4.1 Local Correctness and Global Correctness

Let b be a basis function. Its local correctness can be expressed as a Hoare-style judgment:

$$\{\mathrm{Pre}(b)\}\ b\ \{\mathrm{Post}(b)\}. \tag{9}$$

This means that if the precondition of b holds before execution, then the postcondition of b should hold after execution. The evidence $Evd(b)$ supports this local correctness.

For a complete program structure P composed of multiple basis functions, global correctness requires:

$$\{\mathrm{Pre}(P)\}\ P\ \{\mathrm{Post}(P)\}. \tag{10}$$

The construction problem is therefore to ensure that selected basis functions satisfy their functional positions and that their composition preserves correctness from local units to the whole program.

4.2 Sequential Composition

Sequential composition is the simplest composition form. Suppose two basis functions b_i and b_j are composed so that b_i is executed before b_j. The composition is correctness-preserving only if the result of b_i is sufficient to invoke b_j:

$$Post(b_i) \Rightarrow Pre(b_j). \tag{11}$$

When this condition holds, the composed structure satisfies:

$$\{\mathrm{Pre}(b_i)\}\ b_i; b_j\ \{\mathrm{Post}(b_j)\}. \tag{12}$$

This rule captures many common errors in AI-generated code. For example, a previous step may produce a data structure that does not satisfy the next step's input assumption; an initialization step may be omitted; or a module may guarantee only a weak property while the following module requires a stronger one. These are composition errors rather than surface syntax errors.

4.3 Conditional Branching

For a conditional structure

$$\textbf{if } Cond \textbf{ then } S_1 \textbf{ else } S_2, \tag{13}$$

both branches must satisfy compatible correctness requirements. Given a precondition P and target postcondition Q, the following conditions should hold:

$$\{P \wedge Cond\}\ S_1\ \{Q\}, \tag{14}$$

$$\{P \wedge \neg Cond\}\ S_2\ \{Q\}. \tag{15}$$

The branch conditions should also cover all relevant cases. Otherwise, the constructed program may omit boundary cases or return inconsistent results across branches. In the proposed framework, a branching operator is therefore not only a control-flow construct, but also a condition for preserving postcondition consistency across alternatives.

4.4 Loop Composition

Loops are especially important because many algorithmic errors arise from incorrect iteration orders, missing boundary conditions, or invalid state dependencies. For a loop structure

$$\textbf{while } Cond \textbf{ do } Body, \tag{16}$$

correctness usually depends on an invariant Inv. The loop operator is correctness-preserving when the following conditions hold:

- initialization: $Pre \Rightarrow Inv$;
- preservation: $\{Inv \wedge Cond\}\ Body\ \{Inv\}$;
- exit correctness: $Inv \wedge \neg Cond \Rightarrow Post$;
- termination: a well-founded measure decreases toward termination.

For AI-assisted programming, the most important point is that the loop body cannot be checked independently of the loop operator. A transition basis function may be locally correct only under a particular dependency order. If the surrounding loop violates that order, the composed program is not correct even when the transition formula appears plausible.

4.5 Recursive Composition

Recursive construction requires similar correctness-preserving conditions. A recursive basis function should specify:

- correct base cases;
- a decreasing problem measure or subproblem size;
- valid recursive-call preconditions;
- a result-combination rule that preserves the target postcondition.

These conditions prevent common recursive errors such as missing base cases, non-decreasing recursion, invalid subproblem definitions, or incorrect combination of subproblem results. In the proposed framework, a recursive operator is valid only when these conditions are explicitly satisfied.

4.6 Layered Correctness Evidence

The framework does not require every basis function to carry a full formal proof. Instead, it allows layered correctness evidence. Strong evidence may include machine-checkable proofs, formal verification, or model checking. Medium-strength evidence may include type checking, contract checking, static analysis, or symbolic execution. Weaker evidence may include unit tests, randomized tests, empirical validation, or human review.

This layered view is important for practical AI programming. Critical basis functions and high-risk composition points should use stronger evidence, while lower-risk auxiliary functions may rely on weaker forms of evidence. The key requirement is that the type and strength of evidence should be explicit, so that the construction system can distinguish verified modules from merely tested or reviewed modules.

4.7 Correctness-Preserving Selection

A functional position may have multiple candidate basis functions or candidate program structures. Selection among them should occur only after specification satisfaction has been established. Let

$S(fp_i)$ be a set of candidate structures for functional position fp_i:

$$S(fp_i) = \{S_1, S_2, \dots, S_r\}. \tag{17}$$

If each candidate satisfies the same specification,

$$S_k \vDash Spec(fp_i), \quad k = 1, \dots, r, \tag{18}$$

then an evaluation function F can be used to select a preferred structure:

$$S^* = \arg \min_{S_k \in S(fp_i)} F(S_k). \tag{19}$$

The function F may represent time complexity, space complexity, verification cost, robustness, readability, or engineering preference. This selection step does not create correctness; it chooses among structures whose correctness conditions have already been established.

4.8 Summary

Correct-by-construction AI programming requires more than generating locally plausible code fragments. It requires explicit conditions under which local correctness can be propagated to global correctness. Sequential composition requires postcondition precondition matching; branching requires coverage and postcondition consistency; loops require invariants, dependency consistency, and termination; recursion requires base cases, decreasing subproblems, and correct result combination. These conditions provide the basis for constructing checkable program objects rather than merely generating code strings.

5 Illustrative Case: 0–1 Knapsack

This section illustrates the proposed framework using the 0–1 knapsack problem. The purpose is not to provide an empirical evaluation, but to show how basis-function composition organizes program construction and exposes semantic mismatches that may be missed by direct code generation or limited testing.

5.1 Problem Specification

Given n items, where item i has weight w_i and value v_i, and a knapsack with capacity W, each item can be selected at most once. The goal is to maximize the total value without exceeding the capacity. The specification can be summarized as follows:

- Input: weight array $w[1..n]$, value array $v[1..n]$, and capacity W.
- Precondition: $n \geq 0$, $W \geq 0$, all weights are positive integers, all values are nonnegative, and the two arrays have length n.
- Output: the maximum total value achievable under the capacity constraint.
- Postcondition: the returned value equals the optimal value among all feasible item selections.

At the highest level, the task can be represented as a functional position fp_{KP}. Several candidate basis functions may satisfy this position, including two-dimensional dynamic programming, one-dimensional dynamic programming with a rolling array, memoized search, branch-and-bound, or an integer-programming solver. This shows that a functional position does not uniquely determine an implementation; instead, it defines a specification that can be satisfied by multiple candidate structures.

5.2 Basis-Function Decomposition

For illustration, we select the two-dimensional dynamic programming structure and expand it into lower-level functional positions. The construction can be decomposed into four main basis functions:

- B_{state} defines the state $dp[i][c]$ as the maximum value achievable using the first i items with capacity at most c.
- B_{init} initializes the boundary conditions, including $dp[0][c]$= 0 and $dp[i][0] = 0$.
- B_{trans} computes the transition:

$$\mathrm{dp}[i][c] = \begin{cases} \mathrm{dp}[i-1][c], & \text{if } w_i > c, \\ \max\{\mathrm{dp}[i-1][c], \mathrm{dp}[i-1][c-w_i] + v_i\}, & \text{if } w_i \leq c. \end{cases} \tag{20}$$

- $B_{extract}$ returns $dp[n][W]$ as the final answer.

Each basis function has its own local specification. For example, B_{state} establishes the semantic meaning of the state space; B_{init} guarantees correct base cases; B_{trans} relies on Bellman's principle of optimality; and $B_{extract}$ maps the final state back to the required output. The complete program is constructed only when these basis functions are connected by operators satisfying the required dependency and correctness conditions.

5.3 Correctness Propagation

The two-dimensional dynamic-programming construction provides a simple example of correctness propagation. First, B_{state} defines the semantic meaning of $dp[i][c]$. Second, B_{init} establishes the base cases corresponding to no items or zero capacity. Third, the loop operator applies B_{trans} in an order that respects state dependencies: when computing $dp[i][c]$, the required states $dp[i-1][c]$ and $dp[i-1][c-w_i]$ have already been computed. Fourth, $B_{extract}$ returns $dp[n][W]$, which corresponds to the original problem instance.

Thus, the correctness chain can be summarized as:

$$\begin{aligned} &\text{state semantics} \rightarrow \text{base cases} \rightarrow \text{transition} \\ &\rightarrow \text{dependency-respecting loop order} \rightarrow \text{result extraction}. \end{aligned} \tag{21}$$

This chain shows how local correctness is propagated to global correctness. The program is not treated as a single generated code string. Instead, its correctness is organized through specified construction units and composition conditions.

5.4 Detecting a Composition Mismatch

The same example also shows how the framework can detect semantic mismatches. Consider the one-dimensional rolling-array implementation of 0–1 knapsack. The state $dp[c]$ represents the best value achievable under capacity c after processing the current prefix of items. The transition is:

$$\mathrm{dp}[c] = \max\{\mathrm{dp}[c], \mathrm{dp}[c-w_i] + v_i\}. \tag{22}$$

For the 0–1 version, the capacity loop must iterate in decreasing order:

$$\textbf{for } c = W \textbf{ down to } w_i. \tag{23}$$

The reason is that $dp[c-w_i]$ must still represent the previous layer, before item i is processed. If the loop iterates in increasing order,

$$\textbf{for } c = w_i \textbf{ to } W, \tag{24}$$

then $dp[c - w_i]$ may already have been updated in the same item iteration. In that case, item i can be used more than once, which violates the 0–1 constraint and turns the structure into one closer to unbounded knapsack.

This error is common because the generated code may still be syntactically valid and may even pass simple tests. However, in the proposed framework, the error appears as a composition mismatch: the transition basis function requires old-layer dependency semantics, while the increasing loop operator violates that precondition. The problem is therefore detected not by surface syntax, but by checking whether the selected loop operator satisfies the dependency condition required by the transition basis function.

5.5 Implications

The knapsack example illustrates three features of the framework. First, an algorithmic task can be decomposed into functional positions with explicit specifications. Second, correctness can be propagated through basis functions and composition operators rather than inferred only after code generation. Third, subtle semantic errors can be identified as mismatches between basis functions and their surrounding composition structures.

This example also clarifies the intended role of LLMs. An LLM may help propose the dynamic-programming structure, retrieve candidate implementations, or generate final code. However, correctness depends on whether the state definition, transition rule, loop order, and result extraction satisfy their composition conditions. In this sense, the framework turns AI-assisted programming from direct code generation into structured and checkable program construction.

Although the example is based on dynamic programming, the same construction view can extend to other programming tasks. Graph-search programs, for example, require compatible choices among graph representation, frontier management, visited-set maintenance, and result reconstruction, while data-processing pipelines require compatibility among schema validation, transformation, aggregation, and output formatting. In both cases, semantic errors may appear as mismatches between selected basis functions and their surrounding composition structures. Systematic validation on broader programming tasks remains future work.

6 Discussion and Research Agenda

The basis-function composition framework is intended as a construction-oriented framework rather than a complete implemented system. It clarifies how AI programming can move from direct code generation toward structured and checkable program construction. This section summarizes its applicability, advantages, limitations, and future research directions.

6.1 Applicability

The framework is most suitable for programming tasks with explicit specifications, decomposable structures, and nontrivial correctness requirements, such as algorithmic programs, dynamic programming routines, optimization modules, data processing pipelines, rule-based decision procedures, and high-reliability software components. In such tasks, correctness often depends on state definitions, boundary conditions, dependency orders, invariants, and composition relations, which are difficult to guarantee through direct code generation alone. It is less necessary for low-risk scripts, exploratory prototypes, user-interface fragments, or informal tasks with vague requirements and weak correctness demands, where direct LLM-based code generation followed by testing and human inspection may be more efficient.

State-based optimization domains provide natural testbeds because they contain recurring construction units such as state definitions, transition rules, lookup structures, value-function approximations, request-allocation modules, and rollout policies. Examples include dynamic programming and Markov decision process models, ADP and rollout-based routing policies, and lookup-table designs for efficient state-value representation [19–24].

6.2 Advantages

The framework offers three main advantages. First, it makes program construction explicit by decomposing a task into functional positions, selecting candidate basis functions, checking specifications, and composing them through correctness-preserving operators. Second, it separates generation from correctness: LLMs may assist requirement interpretation, decomposition, candidate retrieval, and code generation, but correctness is supported by specifications, basis-function evidence, and composition conditions. Third, it provides an interface between LLMs and verification-oriented techniques, including type systems, contract checkers, static analyzers, SMT solvers, and proof assistants.

6.3 Limitations

Several limitations remain. The framework depends on basis functions with explicit specifications and correctness evidence, whose construction may be costly. Reliable specification extraction from natural language is also difficult, because user requirements may be incomplete, ambiguous, or inconsistent. In addition, composition checking may become expensive when many candidates and hierarchical expansions are available. Correctness evidence may also be heterogeneous, ranging from formal proofs to tests and human review, making evidence comparison and combination nontrivial. Finally, this paper provides a conceptual framework and an illustrative case rather than a full prototype or empirical evaluation.

6.4 Research Agenda

Future work can proceed in four directions. First, domain-specific basis-function libraries should be developed, recording implementations, interfaces, preconditions, postconditions, evidence types, dependencies, and abstraction levels. Second, retrieval, selection, and pruning mechanisms are needed to rank candidates by specification fit, evidence strength, and engineering constraints while

avoiding exhaustive search. Third, composition checking should be integrated with verification tools such as type checkers, contract systems, static analyzers, symbolic execution engines, SMT solvers, and proof assistants. Fourth, prototype systems and empirical studies are needed to compare basis-function composition with direct LLM generation, generate–test–repair workflows, and human-written implementations in terms of correctness, error types, verification cost, interpretability, runtime performance, code size, and verbosity.

7 Conclusion

This paper presented a basis-function composition framework for correct-by-construction AI programming. The framework addresses a limitation of current LLM-based programming systems: statistical code generation, execution feedback, testing, and repair are useful, but they do not by themselves organize and propagate program correctness during construction. The proposed framework represents a program as a structured composition of basis functions equipped with implementations, preconditions, postconditions, and correctness evidence. Functional positions, candidate basis-function sets, hierarchical expansion, and correctness-preserving composition operators are used to organize program construction before final code realization. Under this view, LLMs assist specification interpretation, candidate retrieval, structural search, and code generation, while correctness is supported by explicit specifications and composition conditions. The 0–1 knapsack example illustrates how correctness can be propagated through state semantics, base cases, transition rules, loop-order dependencies, and result extraction, and how an invalid rolling-array loop order can be identified as a composition mismatch. Overall, verifiable AI programming should move from generating plausible code strings toward constructing structured, interpretable, and checkable program objects.

References

[1] Rajeev Alur et al. 2013. Syntax-Guided Synthesis. In *Proceedings of the Formal Methods in Computer-Aided Design*. IEEE, 1–8. doi:10.1109/FMCAD.2013.6679385

[2] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint:2107.03374* (2021). arXiv:2107.03374 [cs.LG] doi:10.48550/arXiv.2107.03374

[3] Xinyun Chen et al. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2304.05128

[4] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24

[5] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. 2017. Program Synthesis. *Foundations and Trends in Programming Languages* 4, 1–2 (2017), 1–119. doi:10.1561/2500000010

[6] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580. doi:10.1145/363235.363259

[7] Carlos E. Jimenez et al. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. doi:10.48550/arXiv.2310.06770

[8] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated Program Repair. *Commun. ACM* 62, 12 (2019), 56–65. doi:10.1145/3318162

[9] K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (Lecture Notes in Computer Science, Vol. 6355)*. Springer, 348–370. doi:10.1007/978-3-642-17511-4_20

[10] Yujia Li et al. 2022. Competition-Level Code Generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. doi:10.1126/science.abq1158

[11] Aman Madaan et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-2019

[12] M. Douglas McIlroy. 1968. Mass-Produced Software Components. In *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee*, Peter Naur and Brian Randell (Eds.). NATO Scientific Affairs Division, 138–155.

[13] Bertrand Meyer. 1992. Applying "Design by Contract". *Computer* 25, 10 (1992), 40–51. doi:10.1109/2.161279

[14] George C. Necula. 1997. Proof-Carrying Code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. Association for Computing Machinery, 106–119. doi:10.1145/263699.263712

[15] David L. Parnas. 1972. On the Criteria to Be Used in Decomposing Systems into Modules. *Commun. ACM* 15, 12 (1972), 1053–1058. doi:10.1145/361598.361623

[16] Baptiste Rozière et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint:2308.12950* (2023). arXiv:2308.12950 [cs.CL] doi:10.48550/arXiv.2308.12950

[17] Armando Solar-Lezama. 2008. *Program Synthesis by Sketching*. PhD thesis. University of California, Berkeley.

[18] Clemens Szyperski, Dominik Gruntz, and Stephan Murer. 2002. *Component Software: Beyond Object-Oriented Programming* (2 ed.). ACM Press/Addison-Wesley, New York.

[19] Yu Wu and Ming Jian. 2024. State Aggregation and Lower Bound-Based ADP Approach for Dynamic Pick-Up Routing Problem with Capacity Constraint. *IEEE Transactions on Intelligent Transportation Systems* 25, 8 (2024), 9932–9944. doi:10.1109/TITS.2024.3357060

[20] Yu Wu and Xiaoping Qiu. 2024. GRASP-Based Request Allocation and MDP-Based Vehicle Routing in Home Pick-Up Service with Service Continuity Consideration. *IET Intelligent Transport Systems* 18, 2 (2024), 332–345. doi:10.1049/itr2.12454

[21] Yu Wu and Xiaoping Qiu. 2026. A Lookup Table Design Method: Achieving O(1)-Query Complexity and No Memory Waste. In *Theory and Applications of Models of Computation (Lecture Notes in Computer Science, Vol. 16084)*. Springer, Singapore, 63–76. doi:10.1007/978-981-95-4839-2_6

[22] Yu Wu and Bo Zeng. 2023. Dynamic parcel pick-up routing problem with prioritized customers and constrained capacity via lower-bound-based rollout approach. *Computers & Operations Research* 154 (2023), 106176. doi:10.1016/j.cor.2023.106176

[23] Yu Wu, Bo Zeng, and Siming Huang. 2019. A Dynamic Strategy for Home Pick-Up Service with Uncertain Customer Requests and Its Implementation. *Sustainability* 11, 7 (2019), 2060. doi:10.3390/su11072060

[24] Yu Wu, Bo Zeng, and Ming Jian. 2025. ADP- and Rollout-Based Dynamic Vehicle Routing for Pick-Up Service via Budgeting Capacity. *Flexible Services and Manufacturing Journal* 37, 2 (2025), 513–557. doi:10.1007/s10696-024-09551-z

[25] John Yang et al. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-1601

Received 2026-07-17; accepted 2026-08-23